Task-6: - Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to define the requirements for deploying two frontend applications from the provided repository using Apache Web Server with subdomain-based hosting and SSL configuration via **Let's Encrypt**.

This document is intended for DevOps engineers, system administrators, project evaluators, and stakeholders.

1.2 Scope

In Scope:

Deployment of any two frontend applications from:

GitHub Repository: pushkar-iamops/frontend-apps

Configuration of Apache Virtual Hosts

Subdomain-based hosting format:

```
appname-yourname.yourdomain.com
```

SSL implementation using Let’s Encrypt

HTTP to HTTPS redirection

Public accessibility validation

SSL auto-renewal configuration

Out of Scope:

Backend development

Database configuration

CI/CD automation

Load balancing or multi-AZ setup

Advanced monitoring tools

1.3 Definitions, Acronyms, and Abbreviations

Term	Meaning
SSL	Secure Sockets Layer
TLS	Transport Layer Security
Apache	Apache HTTP Web Server
DNS	Domain Name System
HTTP	Hypertext Transfer Protocol
HTTPS	Secure HTTP
Certbot	Let's Encrypt client tool

1.4 References

Apache Official Documentation

Let's Encrypt Documentation

GitHub Repository: pushkar-iamops/frontend-apps

1.5 Overview

This document describes the project overview, functional requirements (limited to three major use cases), non-functional requirements, risk analysis, assumptions, and supporting information for deployment of multiple frontend applications on Apache using SSL and subdomain-based hosting.

---

2. Project Overview

2.1 Project Objectives

Deploy two frontend applications on Apache.

Configure subdomain-based hosting.

Secure applications using Let’s Encrypt SSL.

Enable HTTP → HTTPS redirection.

Ensure public accessibility and secure communication.

2.2 Business Goals

Standardize secure frontend hosting model.

Improve security posture.

Ensure encrypted data transmission.

Enhance user trust and browser compliance.

2.3 Background

Organizations hosting multiple applications on a single server require proper routing and security configuration. Subdomain-based hosting allows logical separation of applications, while SSL ensures encrypted communication between client and server.

This project ensures secure deployment using Apache and trusted SSL certificates.

2.4 Constraints

Time constraint: 4–5 hours

Domain must be pre-purchased

DNS access required

Server must have ports 80 and 443 open

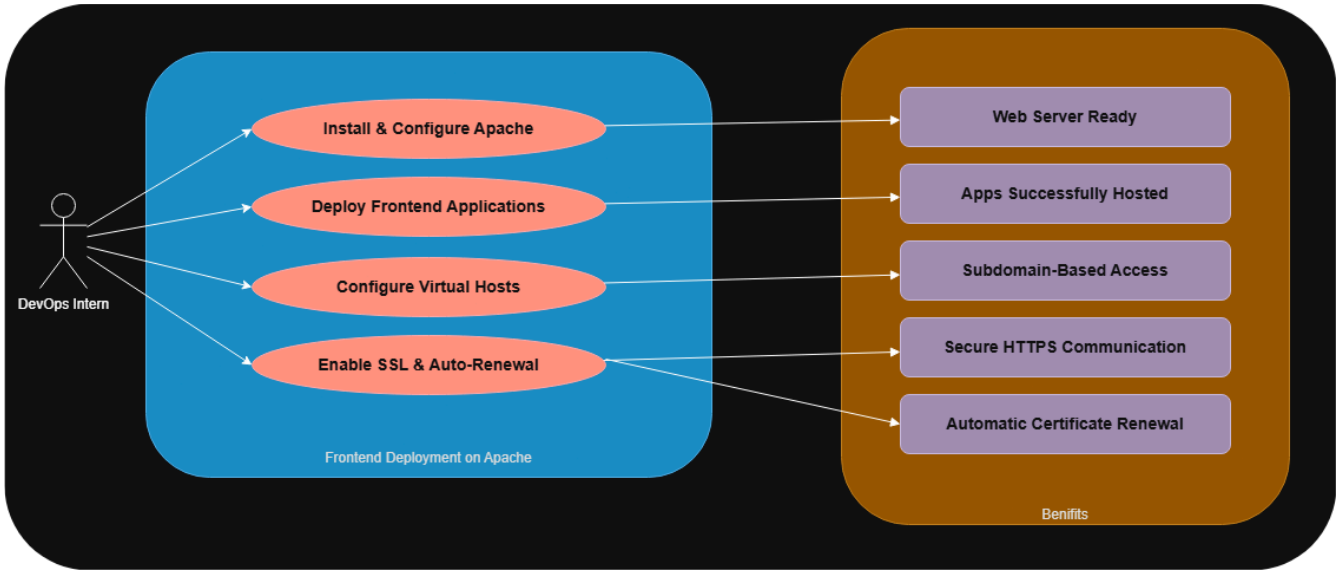
Apache must be used (no Nginx allowed)

---

3. Functional Requirements

3.1 Use Case Diagram

Final use case.png



The following Use Case Diagram represents the functional requirements of the Apache Frontend Hosting System.

The system includes two primary actors:

DevOps Engineer – Responsible for deploying applications and configuring SSL.

End User – Responsible for accessing the deployed applications securely.

The system boundary includes the following major functional activities:

Clone Repository

Configure Apache Virtual Host

Configure DNS Records

Install SSL Certificate

Enable HTTP to HTTPS Redirection

Access Secure Application

All deployment and configuration-related activities are performed by the DevOps Engineer, while the End User interacts only with the deployed application.

(The approved consolidated use case diagram is inserted below this section.)

3.2 Use Cases

3.2.1 Use Case 1: Deploy Frontend Applications on Apache

Description:
The DevOps Engineer selects any two frontend applications and deploys them on Apache using subdomain-based hosting.

Actors:
DevOps Engineer

Preconditions:

Server is provisioned and accessible.

Apache is installed.

Domain and DNS access available.

GitHub repository accessible.

Postconditions:

Applications are accessible via respective subdomains.

Virtual Hosts are properly configured.

Flow of Events:

Clone repository from GitHub.

Select two frontend applications.

Copy application files to Apache document root.

Create separate VirtualHost configuration files.

Enable the sites.

Restart Apache service.

Verify application accessibility via HTTP.

Alternative Flows:

If Apache fails to restart → Validate configuration syntax.

If subdomain not resolving → Verify DNS configuration.

3.2.2 Use Case 2: Configure SSL and Security

Description:

The DevOps Engineer installs SSL certificates and configures HTTP to HTTPS redirection.

Actors:

DevOps Engineer

Preconditions:

Applications deployed successfully.

Subdomains resolving to server IP.

Port 443 open.

Postconditions:

SSL certificates installed successfully.

Applications accessible via HTTPS.

HTTP traffic redirected to HTTPS.

Flow of Events:

Install Certbot.

Request SSL certificates from Let's Encrypt.

Apply certificates to Virtual Hosts.

Enable rewrite rules.

Configure permanent 301 redirection.

Test HTTPS access.

Verify SSL auto-renewal.

Alternative Flows:

SSL validation failure → Check DNS and firewall settings.

Certificate renewal failure → Reconfigure Certbot cron job.

3.2.3 Use Case 3: Access Secure Application

Description:

The End User accesses the deployed frontend application securely via HTTPS.

Actors:

End User

Preconditions:

Deployment completed.

SSL configured successfully.

Postconditions:

Application loads with valid SSL certificate.

Browser displays secure lock icon.

User interacts with application interface.

Flow of Events:

User enters HTTPS URL in browser.

DNS resolves subdomain.

Apache serves application over TLS.

Application loads successfully.

User interacts with frontend features.

Alternative Flows:

If browser shows “Not Secure” → DevOps Engineer verifies SSL configuration.

If application not loading → Check Apache service and firewall settings.

---

4. Non-Functional Requirements

4.1 Performance Requirements

Page load time < 3 seconds

Low latency response

Efficient static file serving

4.2 Security Requirements

Valid SSL certificates

TLS encryption enabled

Port 443 open, unnecessary ports closed

Secure Apache configuration

SSL auto-renewal configured

4.3 Usability Requirements

Clear documentation

Simple configuration process

Easy troubleshooting steps

4.4 Reliability Requirements

Apache restart capability

SSL auto-renewal automation

Server uptime maintained

4.5 Compliance Requirements

Follow Apache best practices

Follow Let’s Encrypt SSL standards

Adhere to secure hosting guidelines

---

5. Assumptions and Dependencies

5.1 Assumptions

Applications are static frontend applications

No backend integration required

Domain is already registered

Public IP available

5.2 Dependencies

DNS provider access

Apache service availability

Let’s Encrypt service availability

Internet connectivity

---

6. Risk Analysis

6.1 Risk Identification

Incorrect DNS configuration

SSL certificate validation failure

Firewall blocking port 443

Apache misconfiguration

SSL expiration

6.2 Risk Mitigation

Verify DNS before SSL request

Test Apache config before restart

Open required ports in firewall

Enable SSL auto-renewal testing

Monitor certificate expiry

7. Glossary

Term	Description

Term	Description
Virtual Host	Apache configuration to host multiple domains
Subdomain	Secondary domain under primary domain
TLS	Encryption protocol securing HTTPS
Certificate Authority	Trusted entity issuing SSL certificates

8. Appendices

Use Case Diagram

Architecture Diagram

SSL Configuration Screenshots

Validation Screenshots

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____